

Software Engineering

Engineering the Development of Software Systems

L06

Software Engineering for AI Systems in Python

Giancarlo Succi

Dipartimento di Informatica – Scienza e Ingegneria
Università di Bologna



Content

- Structure of the Execution in Python
- Interfacing with LLMs
- Langchain
- CrewAI



Structure of the Execution (1/10)

- Python program execution involves three main stages:
 - **Interpreter Initialization**
 - **Compilation** of source code into bytecode
 - **Bytecode Interpretation** by the Virtual Machine
- The CPython interpreter implements all stages internally in C for performance
- Source file processing flow:
 - .py source file → parsed into **Abstract Syntax Tree (AST)**
 - AST → compiled into .pyc **bytecode**
 - Bytecode → executed by the Python **Virtual Machine (VM)**

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (2/10)

- **Interpreter Initialization** prepares the runtime environment by:
 - Loading built-in modules (e.g., `sys`, `builtins`)
 - Setting up memory management and garbage collection
 - Creating the **Global Interpreter Lock (GIL)** for thread safety
- Initialization is done in the `Py_Initialize()` function in CPython

```
import sys
print(sys.builtin_module_names)  # Shows built-in modules loaded at startup
```

- Global interpreter state persists throughout the program execution

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (3/10)

- **Compilation** translates source code into bytecode:
 - Parse source code into an **AST**
 - Convert AST into symbol tables (scopes and variable names)
 - Generate a **code object** containing bytecode and metadata
- The code object represents an executable unit for the VM

```
code = compile("x = 1 + 2", "<string>", "exec")  
print(code.co_code)  # Shows raw bytecode bytes
```

- Compilation logic is implemented in CPython's `compile.c`

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (4/10)

- **Bytecode execution** happens inside the evaluation loop:
 - Main loop implemented in `ceval.c`, function `_PyEval_EvalFrameDefault()`
 - Processes one **frame** at a time
 - Interprets each bytecode instruction sequentially using a dispatch mechanism
- Each **frame** stores:
 - Code object reference (bytecode + metadata)
 - Dictionaries for **locals** and **globals**
 - Operand stack for intermediate values
 - Instruction pointer (program counter)

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (5/10)

- Python maintains a **call stack** of frames:
 - Each function call **pushes** a new frame on the stack
 - Function return **pops** the frame off the stack
 - Frames linked by **f_back** attribute for traceback and call chain

```
import inspect

def foo():
    frame = inspect.currentframe()
    print(frame.f_code.co_name)

foo()  # Outputs: foo
```

- Frames isolate function scopes and support recursion

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (6/10)

- The evaluation loop interprets bytecode **sequentially**
- Common bytecode operations:
 - Load/store variables
 - Arithmetic and logical operations
 - Function calls and returns
 - Control flow and exception handling
- Each bytecode instruction has an **opcode** plus optional operand(s)

```
import dis  
  
def add(x, y):  
    return x + y  
  
dis.dis(add)
```

- Disassembly shows the actual opcodes executed by the VM

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (7/10)

- CPython uses **reference counting** for memory management
 - Each object tracks the number of references to it
 - When count reaches zero, memory is freed immediately
- A cyclic garbage collector detects and collects reference cycles periodically

```
import sys

a = []
print(sys.getrefcount(a)) # Includes temporary references
```

- Hybrid approach balances efficiency and safety

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (8/10)

- The **Global Interpreter Lock (GIL)**:
 - Allows only **one native thread** to execute Python bytecode at once
 - Simplifies memory management by preventing race conditions
 - Limits true parallelism in CPU-bound Python code on multi-core systems
- GIL and thread state are managed via `PyThreadState` structs

```
import threading

print(threading.current_thread().name)  # Usually MainThread
```

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (9/10)

- **C extension modules** can temporarily **release the GIL**:
 - Use macros `Py_BEGIN_ALLOW_THREADS` and `Py_END_ALLOW_THREADS`
 - Allows native threads to run Python code concurrently
 - Enables efficient concurrency in I/O-bound and CPU-bound workloads
- This explains why libraries like NumPy achieve good parallelism despite the GIL

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Execution (10/10)

- Summary:
 - CPython compiles source code into **bytecode**
 - Bytecode executes on a **stack-based virtual machine** via the evaluation loop
 - Call stack frames isolate function calls and variable scopes
 - The **GIL** enforces single-threaded bytecode execution
 - C extensions can bypass the GIL for true parallelism
- Understanding this helps write efficient code and debug complex issues

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Virtual Machine (1/8)

- The Python Virtual Machine (PVM) is a **stack-based interpreter**
- Executes bytecode instructions **one at a time**
- Each instruction operates on a **value stack**
- Designed for simplicity and portability across platforms
- Implemented inside CPython in `ceval.c`
- Core interpreter loop is `_PyEval_EvalFrameDefault()`

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Virtual Machine (2/8)

- The VM manages execution state using **frame objects**
- Each frame stores:
 - Reference to the code object (bytecode + constants + names)
 - Local and global variable dictionaries
 - Operand stack (value stack) for intermediate results
 - Instruction pointer (program counter)
- The VM switches frames during function calls and returns

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Virtual Machine (3/8)

- The **value stack** is central to VM operation:
 - Pushes operands for bytecode instructions
 - Pops results after operation execution
- Example bytecode instructions:
 - `LOAD_CONST` pushes a constant value onto the stack
 - `BINARY_ADD` pops two values, adds them, pushes the result
- The stack-based model simplifies interpreter design and portability

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Virtual Machine (4/8)

- Bytecode instructions consist of:
 - Opcode** — 1 byte representing the operation
 - Optional **operand** — 1 or more bytes depending on the instruction
- The VM decodes opcodes and dispatches execution using:
 - Large **switch** statement or
 - Computed **goto** (a performance optimization)

```
switch(opcode) {  
    case LOAD_CONST:  
        push(consts[operand]);  
        break;  
    case BINARY_ADD:  
        right = pop();  
        left = pop();  
        push(left + right);  
        break;  
    ...  
}
```




Structure of the Virtual Machine (5/8)

- **Fast locals** optimization:
 - Local variables accessed via fixed-size array, not dictionaries
 - Enables faster variable lookup inside function frames
- Code object's `co_varnames` array stores local variable names
- Bytecode instructions like `LOAD_FAST` and `STORE_FAST` use indices into this array

```
LOAD_FAST 0 # Push local variable 0 onto the stack
STORE_FAST 1 # Pop from stack and store in local variable 1
```

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Virtual Machine (6/8)

- Variable namespaces in the VM:
 - `locals()` — current function scope
 - `globals()` — module-level scope
 - `builtins()` — built-in names
- Variable resolution order (LEGB rule):
 - Local
 - Enclosing (for nested functions)
 - Global
 - Built-in

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Virtual Machine (7/8)

- Exception handling is managed by the VM using a **block stack**
- When an exception occurs:
 - The block stack is unwound until an appropriate handler is found
 - Control is transferred to the exception handler or propagates upward
- Important opcodes for exceptions:
 - `SETUP_EXCEPT` — start a try-except block
 - `POP_BLOCK` — end block scope
 - `RAISE_VARARGS` — raise an exception

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Structure of the Virtual Machine (8/8)

- Function calls in the VM use the `CALL_FUNCTION` opcode:
 - Pops the function object and arguments from the stack
 - Creates a new frame for the function call
 - Pushes the new frame on the call stack
 - Starts executing the called function's bytecode
- Returning from the function:
 - Pops the frame off the stack
 - Pushes the return value onto the value stack
- This stack-based design supports nested and recursive calls elegantly

Source: <https://leanpub.com/insidethepythonvirtualmachine/read>



Coding for interfacing with LLMs (1/6)

- Large Language Models (LLMs) like **GPT-4** can be accessed through APIs (Application Programming Interfaces)
- These APIs allows developers to send text input (prompts) and receive generated responses
- **OpenAI Client Library:**
 - Simplifies API usage in Python
 - Handles authentication, request formatting, and response parsing
- General workflow:
 - Import and configure the API client
 - Send a request with a model and a prompt
 - Process the model's output and metadata

Source: <https://platform.openai.com/docs>



Coding for interfacing with LLMs (2/6)

```
from openai import OpenAI

# 1. Initialize client
client = OpenAI()

# 2. Create a chat completion
response = client.chat.completions.create(
    model="gpt-4o-mini", # model selection
    messages=[
        {"role": "system", "content": "You are a helpful tutor."},
        {"role": "user", "content": "Explain recursion simply."}
    ]
)

# 3. Extract the answer
print(response.choices[0].message.content)
```

- `model` specifies which LLM is used
- `messages` define the conversation context
- The response object contains both text and metadata

Source: <https://platform.openai.com/docs>



Coding for interfacing with LLMs (3/6)

- Communication with LLMs is structured into **roles**:
 - system**: defines rules, personality, and behavior
 - user**: instructions or queries from the human
 - assistant**: model responses or intermediate reasoning
- Clear role separation improves reliability and consistency

```
messages = [  
    {"role": "system", "content": "You are a math tutor."},  
    {"role": "user", "content": "What is the integral of  $x^2$ ?"}  
]
```

Source: <https://platform.openai.com/docs>



Coding for interfacing with LLMs (4/6)

- Responses are structured JSON objects
- Contain:
 - `choices[0].message.content`: main text
 - `usage`: tokens consumed (prompt + completion)
 - `finish_reason`: why the generation stopped

```
result = response.choices[0].message.content
print("Answer:", result)

# Access token usage
print("Tokens used:", response.usage.total_tokens)
```

- This information is useful for monitoring costs and limits

Source: <https://platform.openai.com/docs>



Coding for interfacing with LLMs (5/6)

- LLMs can be instructed to call external functions (Tools)
- Functions are described with a JSON schema
- The model decides when to call a function based on user input

```
tools = [  
  {  
    "type": "function",  
    "function": {  
      "name": "get_weather",  
      "description": "Get weather info for a city",  
      "parameters": {  
        "type": "object",  
        "properties": {  
          "city": {"type": "string"}  
        },  
        "required": ["city"]  
      }  
    }  
  }  
]
```



Coding for interfacing with LLMs (6/6)

- Instead of waiting for the full response, stream it token by token
- Useful for:
 - Chat applications (typing effect)
 - Assistants where latency matters

```
with client.chat.completions.stream(  
    model="gpt-4o-mini",  
    messages=[{"role": "user", "content": "Tell a story"}]  
) as stream:  
    for chunk in stream:  
        if chunk.choices[0].delta.content:  
            print(chunk.choices[0].delta.content, end="")
```

Source: <https://platform.openai.com/docs>



LangChain (1/11)

- **LangChain** is a Python framework to build applications powered by Large Language Models (LLMs)
- It solves common challenges: prompt management, state tracking, tool usage, and agent workflows
- Ideal for chatbots, question answering, RAG pipelines, and dynamic workflows
- Without tools like LangChain, you would need to manually coordinate prompt templates, API calls, memory and logic

Source: <https://python.langchain.com/docs/>



LangChain (2/11)

- LLMs require structured and consistent prompts to work reliably
- Complex apps need to maintain context across interactions (e.g. chat history)
- External APIs and databases often need to be queried alongside LLMs
- **Agents** allow the LLM to dynamically choose tools based on user input

Source: <https://python.langchain.com/docs/>



LangChain (3/11)

- PromptTemplate helps create structured prompts with dynamic values
- Use placeholders like `text`, `lang` and define `input_variables`
- Ensures robustness and avoids manual string concatenation

```
from langchain import PromptTemplate

template = "Translate {text} to {lang}."
prompt = PromptTemplate(
    input_variables=["text", "lang"],
    template=template
)

# Usage
prompt.format(text="Hello", lang="Italian")
```

Source: <https://python.langchain.com/docs/>



LangChain (4/11)

- **LLMChain** supports the combination of a prompt for an LLM and its submission to it (e.g. OpenAI)
- Handles API communication and response parsing
- Acts like a pipeline: input → prompt → model → output

```
from langchain.llms import OpenAI
from langchain import LLMChain

llm = OpenAI(model_name="gpt-4")
translator_chain = LLMChain(llm=llm, prompt=prompt)

result = translator_chain.run({"text": "Hi", "lang": "French"})
print(result)  # Bonjour
```

Source: <https://python.langchain.com/docs/>



LangChain (5/11)

- **Memory** enables the model to remember past inputs/outputs
- Types:
 - **ConversationBufferMemory**: stores raw chat history
 - **ConversationSummaryMemory**: stores summarized history
- Useful for chatbots and contextual reasoning

```
from langchain.memory import ConversationBufferMemory
from langchain import LLMChain, PromptTemplate
from langchain.llms import OpenAI

llm = OpenAI()
prompt = PromptTemplate(
    input_variables=["history", "question"],
    template="Conversation so far:\n{history}\n\nUser: {question}\nAnswer:"
)
memory = ConversationBufferMemory(memory_key="history")
chat_chain = LLMChain(llm=llm, prompt=prompt, memory=memory)
print(chat_chain.run({"question": "Hello, my name is Alice"}))
print(chat_chain.run({"question": "What's my name?"}))
# Second call will recall "Alice" thanks to memory
```



LangChain (6/11)

- In Python it is possible to use functions with LLMs. The mechanism to wrap functions inside LLMs is called **Tools**
- Agents decide when and how to call them during reasoning
- Examples: search engine, calculator, file access, custom APIs

```
from langchain.agents import Tool

def search_web(query):
    return f"Search results for: {query}"

tools = [                                # Wrap the function as a LangChain Tool
    Tool(
        name="Search",
        func=search_web,
        description="Useful for answering questions about the web"
    )
]

print(tools[0].func("capital of Spain"))    # Example use (later by an Agent)
# -> "Search results for: capital of Spain"
```




LangChain (7/11)

Agents are components that use an LLM to decide what actions to take next

- An agent reasons step by step:
 - Receives a user input
 - Decides if it needs a tool (e.g., calculator, search API)
 - Executes the tool with intermediate reasoning
 - Uses the result to generate the final answer
- Agents are powered by an LLM that interprets the context and dynamically selects tools

Source: <https://python.langchain.com/docs/>



LangChain (8/11)

An agent calls functions following these steps:

- **Prompt & tools:** the LLM sees tool names and descriptions, the conversation and a *scratchpad*
- **Cycle:**
 - *Thought* → decide action
 - *Action* → pick Tool(name, args)
 - *Observation* → tool's result
 - Repeat until *Final Answer*
- **AgentExecutor** glues everything: parses the LLM's action, executes the tool, appends observation, loops

Source: <https://python.langchain.com/docs/>



LangChain (9/11)

- Tools are Python callables with names, docstrings and typed params
- Type hints help the LLM choose and format arguments

```
from langchain.tools import tool

@tool
def search_web(query: str) -> str:
    """Search the web for up-to-date facts."""
    return f"Results for '{query}'"

@tool
def multiply(x: float, y: float) -> float:
    """Multiply two numbers."""
    return x * y

tools = [search_web, multiply] # passed to the agent
```

Source: <https://python.langchain.com/docs/>



LangChain (10/11)

- By using a chat model and a ReAct prompt, AgentExecutor shows the loop (`verbose=True`).

```
from langchain_openai import ChatOpenAI
from langchain import hub
from langchain.agents import create_react_agent, AgentExecutor

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
prompt = hub.pull("hwchase17/react") # ReAct prompt template

agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

executor.invoke({
    "input": "What's the population of Italy? Then multiply it by 2."
})

# Console shows: Thought Action (tool+args) Observation ... Final Answer
```

Source: <https://python.langchain.com/docs/>



LangChain (11/11)

- Some chat models emit *tool calls* (name+args) directly.
- Bind tools to the model, execute calls, feed results back.

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
llm_with_tools = llm.bind_tools([multiply]) # expose schema to the LLM

msg = llm_with_tools.invoke("Please multiply 12 by 7.")
if msg.tool_calls:
    for tc in msg.tool_calls:
        if tc["name"] == "multiply":
            result = multiply.invoke(tc["args"]) # run Python function
            #In an agent, the result becomes an Observation and the loop continues
```

Source: <https://python.langchain.com/docs/>



CrewAI (1/9)

- **CrewAI** is an open-source framework to build multi-agent systems in Python
- Inspired by real teamwork:
 - each agent has a role, tools, and memory
 - agents collaborate inside a crew
- Designed for:
 - task delegation
 - modular workflows
 - reasoning across multiple steps

Source: <https://docs.crewai.com/en/introduction>



CrewAI (2/9)

- LLMs are strong at understanding text, but weak at:
 - **Planning:** managing multi-step tasks
 - **Memory:** retaining context across steps
 - **Tool use:** interacting with APIs or systems
- CrewAI structures LLMs into agents that cooperate
- Makes large tasks decomposable and interpretable

Source: <https://docs.crewai.com/en/introduction>



CrewAI (3/9)

The core elements of this functioning are

- **Crew:** a team of agents working together
- **Agent:** an autonomous LLM with:
 - role (e.g. Planner, Researcher)
 - tools (functions/APIs it can use)
 - memory (optional)
- **Task:** unit of work assigned to one agent
- Agents can reason, use tools, delegate, and share results

Source: <https://docs.crewai.com/en/introduction>



CrewAI (4/9)

- Each agent is defined with a role, goal, and LLM backend
- Example: a Researcher Agent*

```
from crewai import Agent
from langchain_openai import ChatOpenAI

# 1. Define the LLM backend (can be OpenAI, Anthropic, etc.)
llm = ChatOpenAI(model="gpt-4o-mini")

# 2. Create an Agent with a role and purpose
researcher = Agent(
    role="Researcher",      # High-level function in the team
    goal="Find reliable information on AI trends",  # Explicit objective
    backstory="Expert in AI news and technical reports.",  # Context that guides
    reasoning
    llm=llm      # Language model powering the agent
)

# This agent can now be assigned tasks and will act autonomously within a Crew
```

Source: <https://docs.crewai.com/en/introduction>



CrewAI (5/9)

- A task is a unit of work assigned to an agent
- *Example*: collect key fashion trends in 2025

```
from crewai import Task

# 1. Define a specific unit of work for an Agent
task = Task(
    description="Summarize the top 3 fashion trends for 2025.",

    expected_output="A bullet list with sources.",
    # Defines the required format of the output

    agent=researcher
    # Assigns the task to the 'researcher' Agent defined earlier
)

# This Task can be added to a Crew
# When executed, the assigned Agent will use its role, goal, and LLM to complete
  the work and return the requested format
```

Source: <https://docs.crewai.com/en/introduction>



CrewAI (6/9)

- A **Crew** is a group of agents and tasks
- It organizes execution and dependencies

```
from crewai import Crew

# 1. Create a Crew: a team that coordinates Agents and Tasks
crew = Crew(
    agents=[researcher],      # List of agents in the crew
    tasks=[task],             # Tasks that the crew must complete
    verbose=True              # Enables detailed logging of the execution steps
)

# 2. Start the execution of the crew
result = crew.kickoff()      # The crew assigns tasks to agents and collects
                             results

# 3. Print the final output produced by the Agent
print(result)
```

Source: <https://docs.crewai.com/en/introduction>



CrewAI (7/9)

- Crews can include multiple agents with different roles
- *Example:* Planner + Researcher + Writer

```
planner = Agent(role="Planner", goal="Break tasks down", llm=llm)
writer = Agent(role="Writer", goal="Draft final report", llm=llm)

crew = Crew(
    agents=[planner, researcher, writer],
    tasks=[task],
    verbose=True
)
```

Source: <https://docs.crewai.com/en/introduction>



CrewAI (8/9)

- Tasks can be executed in different modes:
 - **Sequential execution:** tasks are processed one after another, where the output of task A becomes the input for task B, like a pipeline
 - **Parallel execution:** independent tasks run simultaneously, improving efficiency for large workflows
- The Crew automatically:
 - Manages task **dependencies**, ensuring order when needed
 - Preserves **context** across tasks, sharing memory and results
 - Assigns the right **agent** to each task based on its role and goal

Source: <https://docs.crewai.com/en/introduction>



CrewAI (9/9)

- **LangChain:**

- Focus on tooling and pipelines: prompt templates, memory, retrievers, agent executors
- Provides the building blocks for LLM-powered applications
- Great for integrating data sources, APIs, and customizing reasoning flows

- **CrewAI:**

- Focus on multi-agent coordination: simulating real-world teams
- Each agent has a role, goal, backstory, memory and tools
- Designed for collaborative tasks, delegation, and workload distribution

Source: <https://docs.crewai.com/en/introduction> <https://python.langchain.com/docs/>



Questions?

End of the Lectures on Software Engineering for
AI Systems in Python.